CSIE 5452, Spring 2026: Homework 1

B13902022 Yu-Chi,Lai

1 Timing Analysis of the CAN Protocol: Part I (12pt)

1.1

Firstly, set $Q_0 = B_0 = C_1 = 30$, since no other task has higher priority than μ_0, the terms in the summation is zero.

Hence, the worst-case response time $R_0 = Q_0 + C_0 = 30 + 10 = 40$ (msec).

Iteration	LHS (Q_0)	B_0	RHS	Stop?
1	30	30	30	yes

1.2

Let $Q_1 = B_1 = C_1 = 30$ as the beginnning.

Iteration	LHS (Q_1)	B_1	j	$Q_1 + \tau$	T_j	$\lceil \frac{Q_1+\tau}{T_j} \rceil$	C_j	RHS	Stop?
1	30	30	0	30.1	50	1	10	40	no
2	40	30	0	40.1	50	1	10	40	yes

Thus, the worst-case response time of μ_1 is $R_1 = Q_1 + C_1 = 40 + 30 = 70$ (msec).

1.3

Let $Q_2 = B_2 = C_2 = 20$ as the beginnning. After 3 iterations, Q_2 converges. Hence, the worst case response

Iteration	LHS (Q_2)	$\lceil \frac{Q_2+\tau}{T_0} \rceil C_0 + \lceil \frac{Q_2+\tau}{T_1} \rceil C_1$	RHS	Stop?
1	20	40	60	no
2	60	50	70	no
3	70	50	70	yes

time $R_2 = Q_2 + C_2 = 70 + 20 = 90$ (msec).

2 Timing Analysis of the CAN Protocol: Part II (36pt)

2.1

The answers below are calculated using the code in 2.2.

$$
\begin{aligned}
R_0 &= 1.44 \\
R_1 &= 2.04 \\
R_2 &= 2.56 \\
R_3 &= 3.16 \\
R_4 &= 3.68 \\
R_5 &= 4.28 \\
R_6 &= 5.20 \\
R_7 &= 8.40 \\
R_8 &= 9.00 \\
R_9 &= 9.68 \\
R_{10} &= 10.20 \\
R_{11} &= 19.36 \\
R_{12} &= 19.80 \\
R_{13} &= 20.32 \\
R_{14} &= 29.40 \\
R_{15} &= 29.76 \\
R_{16} &= 30.28
\end{aligned}
$$

2.2

```cpp
#include <bits/stdc++.h>
using namespace std;

signed main() {
        ifstream fin("input.dat");
        int n;
        double tau;
        fin >> n;
        fin >> tau;
        vector<int> priority(n);
        vector<double> transmission_time(n), period(n);
        for (int i = 0; i < n; ++i) {
                fin >> priority[i] >> transmission_time[i] >> period[i];
        }
        fin.close();

    vector<double> res(n);
    for (int i = 0; i < n; i++) {
        double block = 0;
        for (int j = 0; j < n; j++) {
            if (priority[j] >= priority[i]) {
                block = max(block, transmission_time[j]);
            }
        }

        double wait = block;
```

```
        while (1) {
            double result  = block;
            for (int j = 0; j < n; j++) {
                if (priority[j] < priority[i]) {
                    result += transmission_time[j] * ceil((wait + tau) / period[j]);
                }
            }
            if (result > period[i]) {
                res[i] = -1;
                break;
            }
            if (result-wait <= 1e-9 && result-wait >= -(1e-9)) {
                res[i] = transmission_time[i] + wait;
                break;
            } else {
                wait = result;
                continue;
            }
        }
    }

    for (int i = 0; i < n; i++) {
        cout << res[i] << endl;
    }
}
```

3 Timing Analysis of Preemptive Fixed-Priority Scheduling (16pt)

3.1

Iteration	LHS (R_0)	C_0	RHS	Stop?
1	10	10	10	yes

3.2

Iteration	LHS (R_1)	C_1	j	R_1	T_j	$\lceil \frac{R_1}{T_j} \rceil$	C_j	RHS	Stop?
1	30	30	0	30	50	1	10	40	no
2	40	30	0	40	50	1	10	40	yes

3.3

Iteration	LHS (R_2)	C_2	j	R_2	T_j	$\lceil \frac{R_2}{T_j} \rceil$	C_j	RHS	Stop?
1	20	20	0	20	50	1	10	60	no
			1		200	1	30		
2	60	20	0	60	50	2	10	70	no
			1		200	1	30		
3	70	20	0	70	50	2	10	70	yes
			1		200	1	30		

3.4

Although preemptive fixed-priority scheduling is often disadvantageous to the lowest-priority task because higher-priority tasks can interrupt it multiple times, in this case τ_2 still has a smaller worst-case response time than μ_2.

The key reason is that non-preemptive CAN analysis includes an additional blocking term B_2:

$$Q_2 = B_2 + \sum_{j:P_j<P_2} \left\lceil \frac{Q_2 + \tau}{T_j} \right\rceil C_j, \qquad R_{\mu_2} = Q_2 + C_2.$$

In contrast, preemptive fixed-priority analysis does not have this blocking term:

$$R_{\tau_2} = C_2 + \sum_{j:P_j<P_2} \left\lceil \frac{R_{\tau_2}}{T_j} \right\rceil C_j.$$

From the computed results, $R_{\mu_2} = 90$ msec while $R_{\tau_2} = 70$ msec. Therefore, for this task set, the reduction from removing non-preemptive blocking is larger than the extra delay caused by preemptions, so τ_2 ends up with a smaller worst-case response time.

4 Timing Analysis of TDMA-Based Protocols (12pt)

4.1

$$(4, 10, 1, 2, 6, 7)$$

4.2

$$(4, 10, 0, 3, 5, 6, 10, 13, 15, 16)$$

4.3

$$(4, 10, 1, 2, 6, 7, 11, 12, 16, 17)$$

4.4

k	$\max_{1\le j\le n}(s_{j+k} - s_j)$	$\min_{1\le i\le m}(a_{i+k-1} - a_i)$	(Column-2) - (Column-3)
1	4	0	4
2	5	1	4
3	9	3	6
4	10	6	4

4.5

$$6 + 1 = 7$$

The worst case response time is 7.

5 MILP Linearization (12pt)

5.1

α	β	γ	LHS	$\alpha+\beta-\gamma\le 1$	$\alpha-\beta+\gamma\le 1$	$-\alpha+\beta+\gamma\le 1$	RHS	LHS=RHS?
0	0	0	T	T	T	T	T	yes
0	0	1	T	T	T	T	T	yes
0	1	0	T	T	T	T	T	yes
0	1	1	F	T	T	F	F	yes
1	0	0	T	T	T	T	T	yes
1	0	1	F	T	F	T	F	yes
1	1	0	F	F	T	T	F	yes
1	1	1	T	T	T	T	T	yes

5.2

α	β	γ	LHS	$\alpha+\beta-1\le\gamma$	$\gamma\le\alpha$	$\gamma\le\beta$	RHS	LHS=RHS?
0	0	0	T	T	T	T	T	yes
0	0	1	F	T	F	F	F	yes
0	1	0	T	T	T	T	T	yes
0	1	1	F	T	F	T	F	yes
1	0	0	T	T	T	T	T	yes
1	0	1	F	T	T	F	F	yes
1	1	0	F	F	T	T	F	yes
1	1	1	T	T	T	T	T	yes

5.3

Choose the smallest feasible big-M as

$$M = 2026.$$

To guarantee

$$\alpha x = y \iff 0 \le y \le x \ \wedge\ x - M(1-\alpha) \le y \ \wedge\ y \le M\alpha,$$

we analyze both values of α. (Also the relationship given by the problem is trivially bidirectional.)

If $\alpha = 0$, then the RHS becomes

$$0 \le y \le x \wedge x - M \le y \wedge y \le 0.$$

From $0 \le y$ and $y \le 0$, we get $y = 0$, matching the LHS ($\alpha x = y \Rightarrow 0 = y$). For feasibility, we also need $x - M \le 0$ for all valid x, i.e., $M \ge x$. Since $x \le 2026$, this requires $M \ge 2026$.

If $\alpha = 1$, then the RHS becomes

$$0 \le y \le x \wedge x \le y \wedge y \le M.$$

From $y \le x$ and $x \le y$, we get $y = x$, matching the LHS ($\alpha x = y \Rightarrow x = y$). To avoid excluding valid x, we need $x \le M$ for all $x \le 2026$, so again $M \ge 2026$.

Therefore, any $M \ge 2026$ works, and the tightest choice is

$$\boxed{M = 2026}.$$

6 Signal Packing (12pt)

6.1

The number of bits which need to be transmitted per message is the data field length plus 47. In original scheme, μ_0's data field length need to be padded to 8 and μ_1's need to be padded to 16. Hence, the total transmitted bits is $(8 + 47) + (16 + 47) = 118$.

In the second scheme, the length of data field of μ_0' doens't need to pad since it's already the multiple of 8. So the total number of bits needed to transmit is $16 + 47 = 63$, which is much better than the first scheme regarding number of bits. (The periods and μ_2 and μ_3 part are unchanged, so we don't need to calculate the sum of all messages again.)

6.2

No, μ_0' and μ_2 are sent from different ECUs.

6.3

Yes, we can merge μ_3 into μ_0', it becomes μ_0''. Our new messaging scheme can be shown as below.

Message	Sender	Receiver(s)	Number of Bits (Data Field)	Period (msec)
μ_0''	ε_0	$\varepsilon_1, \varepsilon_3$	32	50
μ_2	ε_1	$\varepsilon_2, \varepsilon_3$	10	50

where the first 6 bits of μ_0'' are the bits from μ_0 and the following 10 bits ofμ_0'' are the bits from μ_1, and the rest of it (16 bits) is from μ_3. (Refer the first table given by problem)

This way, we can further improve the bit efficiency compared to the second scheme. Within 100 ms, the length of data field of μ_0'' is 32 and sent twice, hence the number of bits transmitted is $(32 + 47) \times 2 = 158$ (bits). While sending μ_3 and μ_0' respectively cost $(16 + 47) + (16 + 47) \times 2 = 189$ (bits).

Hence, we further improve the second scheme without breaking any policy (Only μ_3 in original scheme is sent more frequently.)